6. **Working MVP beats ambitious half-system.** At Week 12, the loop must work. Cut everything else if it does not.
7. **Scope creep is the enemy.** New ideas go in `docs/ideas.md`, not into the build.

12. What you do on Monday

1. Create the GitHub repo with the folder structure.
2. Write `README.md` with the one-line pitch and the free stack.
3. Write `docs/plan.md` — paste this consolidated plan into it.
4. Start Week 1: Python fundamentals, virtual environment, first FastAPI `/health` route.
5. Commit before you sleep.

Everything else follows the plan. You have 16 weeks, a decided stack, a decided schema, a decided evaluation, and a decided demo. The only thing left is to build it.

You are ready. Start Week 1.

> now give me full plan why we making this what is research we done . and all other planing and all now give me all in depth give each thing in depth . this we use for our project this is most critical so make it carefully and perfect and dont skip anything

Thought for 2 seconds

> The user wants a complete, in-depth, consolidated plan document for their major project. They want everything: why we're making this, what research we've done, all planning, all in depth. This is the most critical document for their project. They explicitly said "don't skip anything" and "make it carefully and perfect."
>
> Let me compile everything from the entire conversation into one comprehensive, well-structured document. I need to cover:
>
> 1. Project title and identity
> 2. Motivation / why we're building this (the problem)
> 3. Research questions and novelty
> 4. Literature review / research context
> 5. The vertical (service-based consulting)
> 6. Teams and access levels
> 7. Dataset design
> 8. Graph schema
> 9. Extraction pipeline
> 10. Architecture (backend, frontend, stack)
> 11. Semester 1 16-week plan
> 12. Semester 2 plan
> 13. Evaluation methodology
> 14. Risk and fallback plan
> 15. Demo script
> 16. Report structure

17. Business roadmap
18. MCP placement
19. Rules and principles

Let me write this as a single, comprehensive document that the user can reference throughout their project. I'll make it detailed but well-organized with clear sections. No shortcuts. Every term explained where needed. This should be the definitive reference document.

Let me be thorough and careful. This is essentially a project charter + technical design document + execution plan all in one.

I'll structure it clearly with numbered sections so they can navigate it easily. I'll make it long but organized, because they explicitly asked for depth and "don't skip anything."

COMPLETE MAJOR PROJECT PLAN — NOVA CONSULTING MEMORY AGENT

Version 1.0 — Definitive Reference Document
Use this for: project proposal, semester reviews, report writing, demo day, and resume claims.

SECTION 1: PROJECT IDENTITY

1.1 Title

A Self-Growing Organizational Memory Agent using Graph Retrieval and Expert Routing for Service-Based Consulting Firms

1.2 Short title (for slides and resume)

Nova Memory Agent — Team memory that answers, admits ignorance, routes to experts, learns from human replies, and reasons over a knowledge graph.

1.3 One-paragraph description

Service-based consulting companies lose knowledge continuously. Proposals, delivery lessons, support fixes, and expert know-how are scattered across documents, chat messages, and people's heads. When someone leaves or forgets, the company repeats mistakes and rewrites proposals from scratch. Nova Memory Agent is a system that ingests the company's documents and discussions, answers team questions with citations, admits when it does not know, routes the question to the right human expert, saves that human's answer as permanent memory, and uses a knowledge graph to answer multi-hop questions that standard retrieval-augmented generation fails on. The system is evaluated against standard retrieval baselines on a 120-question benchmark, and the evaluation is the research contribution.

1.4 Elevator pitch (10 seconds)

"Most AI tools either guess or wait to be asked. Ours admits when it does not know, finds the right person, learns from their answer, and uses a graph to answer 'why' questions that normal search cannot."

SECTION 2: WHY WE ARE BUILDING THIS (THE PROBLEM)

2.1 The core problem

A consulting company does projects for clients. Every project produces knowledge: proposals, pricing, technical approaches, mistakes, fixes, and lessons. This knowledge is the company's most valuable asset after its people.

But this knowledge is not captured systematically. It lives in:

- Old proposal documents in shared drives with names like `Proposal_2024_final_v3.docx`
- Delivery notes in project management tools
- Support tickets in ticketing systems
- Slack or Teams messages that disappear into history
- The memories of senior people who eventually leave

2.2 The three concrete failures

Failure 1: Proposal reuse fails.
A pre-sales manager needs to write a proposal for a "legacy ERP to cloud migration." A nearly identical project was done 18 months ago. The old proposal has the correct pricing, timeline, approach, and documented risks. But the manager cannot find it. They spend 3 days writing from scratch, quote a wrong price, and miss a known risk. Cost: 2 lost days + underpriced deal + client dissatisfaction.

Failure 2: Mistakes repeat.
A delivery team starts a "real-time consent tracking" feature for a healthcare client. Eighteen months ago, another team tried the same thing for a different client. It failed because the client's legacy system could not handle real-time processing; the team switched to batch processing after 3 wasted weeks. The old lesson exists in a post-mortem PDF that nobody reads. Cost: 3 wasted weeks + delayed delivery.

Failure 3: Expertise is invisible.
A support engineer faces a "nightly server restart" incident. A colleague solved the exact issue 6 months ago and wrote a runbook. But nobody knows who that colleague is or where the runbook is. The support engineer debugs from scratch for 2 hours. Cost: 2 hours per incident × many incidents × many clients.

2.3 Why existing tools fail

Existing tool	Why it fails
Wiki / Confluence	Nobody maintains it. It rots. Search is keyword-based and misses meaning.
Shared drives	Files are found only if you know the filename. No semantic search. No connections between documents.
ChatGPT / generic LLM	Does not know your company. Hallucinates when it does not know. Cannot cite sources.
Standard "chat with your docs" RAG	Answers single-fact questions but fails on multi-hop questions ("which clients had data quality issues and who led those projects?"). Cannot admit ignorance reliably. Cannot route to experts. Cannot learn from human answers.
Notion AI / Glean / enterprise search	Expensive. Keyword + basic semantic search. No knowledge graph reasoning. No self-growing memory. No expert routing. No multi-hop.

2.4 What is genuinely missing

No existing open tool combines all five of these:

1. **Self-growing memory** — every human answer becomes permanent, cited team knowledge automatically
2. **Honest ignorance** — the system admits when it does not know instead of hallucinating
3. **Expert routing** — when it does not know, it finds the person who does
4. **Knowledge graph reasoning** — answers multi-hop questions by following connections, not just text similarity
5. **Measurable comparison** — proven better than standard RAG on a real benchmark

That combination is the project.

SECTION 3: RESEARCH CONTRIBUTION AND QUESTIONS

3.1 The research gap

Standard Retrieval-Augmented Generation (RAG) works by converting text to vectors (numeric representations) and finding text with similar vectors. It fails on questions whose answer requires connecting facts across multiple documents.

Example multi-hop question: *"Which clients had data quality problems, and who led those projects?"*

- Standard RAG finds documents about "data quality" and documents about "projects" but cannot connect "client → project → problem" and "project → person" in one answer.
- GraphRAG retrieves along graph edges: Client → FOR_CLIENT → Project → HAD_PROBLEM → Problem, then Project → WORKED_ON → Person. The answer follows the path.

This difference has been studied for general knowledge graphs, but not specifically for **organizational memory in service-based consulting firms** with **expert routing** and **self-growing memory**. That is the gap.

3.2 Research questions

RQ1 (Primary): Does GraphRAG improve answer accuracy on multi-hop organizational questions compared to standard vector RAG?

- Hypothesis: GraphRAG achieves ≥ 65% accuracy on multi-hop questions vs ≤ 40% for standard RAG.
- Metric: Multi-hop Accuracy on a 30-question multi-hop subset.

RQ2: Does expert routing reduce the rate of hallucinated answers compared to a standard RAG system that always answers?

- Hypothesis: Expert routing reduces hallucination rate from ~9% (standard RAG) to ≤ 5%.
- Metric: Hallucination Rate on unanswerable questions.

RQ3: Does the self-growing memory loop (saving human answers) improve accuracy over time?

- Hypothesis: After 20 saved human answers, accuracy on those question types improves by ≥ 15 percentage points.
- Metric: Before/after accuracy on a held-out question set covering saved topics.

RQ4 (Secondary): Does hybrid graph + vector retrieval with reranking outperform vector-only retrieval on decision-why questions?

- Hypothesis: Hybrid retrieval improves decision-why accuracy by ≥ 20 percentage points.
- Metric: Decision Recall and Answer Accuracy on the 20-question decision subset.

3.3 Novelty claims (honest)

What is genuinely new in this work:

1. **A self-growing memory architecture** where human answers become graph facts automatically, not just documents in a vector store.
2. **Integration of expert routing into a GraphRAG pipeline** — most GraphRAG work does not route to humans.
3. **A domain-specific dataset** (Nova Consulting) that captures realistic consulting knowledge: proposals, delivery notes, support tickets, client confidentiality.
4. **A systematic comparison** of LLM-only, vector RAG, hybrid RAG, and full GraphRAG+expert-routing on organizational memory questions.

What is NOT claimed as new (be honest in the report):

- GraphRAG itself is not new (Microsoft Research, 2024).
- RAG is not new (Lewis et al., 2020).
- Expert routing and human-in-loop are not new.
- The novelty is the combination, the domain, and the evaluation.

3.4 Relation to existing work

Key papers to cite (minimum 15–25 in the report):

- **RAG:** Lewis et al., 2020 — "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"
- **GraphRAG:** Edge et al., Microsoft Research, 2024 — "From Local to Global: A Graph RAG Approach to Query-Focused Summarization"
- **Self-RAG:** Asai et al., 2023 — self-reflective retrieval
- **Organizational memory:** Walsh & Ungson, 1991 — foundational organizational memory theory
- **Knowledge graphs in enterprises:** Hogan et al., 2021 — "Knowledge Graphs" survey
- **Reranking:** Nogueira & Cho, 2019 — passage reranking with transformers
- **Human-in-loop:** Monarcha et al., 2021 — interactive machine learning
- **Agentic RAG:** recent surveys on agentic retrieval
- **Evaluation:** RAGAS, TruLens, and LLM-as-judge methodology

Action for you: read the first three carefully. The rest can be skimmed for citations.

SECTION 4: THE VERTICAL — SERVICE-BASED CONSULTING

4.1 Why service-based, not product-based

Criterion	Service-based	Product-based
Knowledge fragmentation	Very high (proposals, delivery, support, people)	Lower (mostly GitHub)
Reuse value	Very high (proposals reused across clients)	Medium
Confidentiality problem	Central (Client A vs Client B)	Minor
Novelty for research	Higher (less explored)	Lower (many student projects on GitHub Q&A)
Public data availability	Harder (simulate)	Easy (GitHub)
GraphRAG use case	Clear multi-hop across entities	Less clear

Decision: service-based. The confidentiality + reuse problem is genuinely novel and gives the project a research edge. Public data is harder, but the simulated dataset is itself a contribution.

4.2 The fictional company: Nova Consulting

- Industry: IT and management consulting
- Size: ~15 employees (small enough to simulate honestly, large enough to be realistic)
- Clients: 5–6, spanning healthcare, retail, manufacturing, finance
- Projects: 8–10, spanning 2024–2025
- Teams: Pre-Sales, Delivery, Support

4.3 Teams and roles

Team 1: Pre-Sales (4 people)

- **Job:** Talk to clients before a project starts, write proposals, estimate price and time, win the deal.
- **Needs:** Find similar past proposals, past pricing, past case studies, past experts.
- **Loses without the system:** Starts from zero every time, quotes wrong prices, misses known risks.
- **Roles inside team:** Analyst, Pre-Sales Manager.

Team 2: Delivery (8 people)

- **Job:** Actually build and deliver the project after the client agrees.
- **Needs:** Find past technical approaches, warnings about what went wrong, who solved similar problems, save new lessons.

- **Loses without the system:** Repeats mistakes, wastes weeks, new people struggle.
- **Roles inside team:** Engineer, Senior Engineer, Tech Lead, Delivery Manager.

Team 3: Support (3 people)

- **Job:** Handle problems after delivery — incidents, bugs, client issues.
- **Needs:** Find past incidents and fixes, runbooks, who fixed similar issues, save new fixes.
- **Loses without the system:** Solves the same problem repeatedly, slow client response.
- **Roles inside team:** Support Engineer, Support Lead.

Team 4: Admin (system team)

- **Job:** Manage the system itself — configure data sources, manage users, approve memory, control access.
- **Who:** One or two senior people (in your project, you).
- **Needs:** Add/remove data sources, approve or delete saved answers, manage users, see analytics.

4.4 Access levels (5 levels)

Level	Name	What they can do
1	**Admin**	Everything. Configure system, manage users, delete memory, see all teams.
2	**Manager**	See their team's data, approve saved answers, see team analytics. Cannot configure system.
3	**Senior Member**	Save answers, correct wrong answers, ask, see team data. Cannot manage users.
4	**Member**	Ask, view answers, react to save, see team data. Cannot save directly.
5	**Guest**	Ask only, see public answers only.

4.5 Data separation (confidentiality rules)

Data type	Who can see it
Public case studies	Everyone
Anonymized lessons learned	Everyone
Proposals with client names and pricing	Pre-Sales + Admin only

Data type	Who can see it
Delivery notes with client specifics	Delivery + Admin only
Support tickets with client data	Support + Admin only
Cross-team insights (no client names)	Managers and above, all teams
Expertise map (who knows what)	Everyone

Anonymization rule: remove client name, remove personal details, keep the lesson. Example: "A healthcare client's legacy system could not handle real-time consent tracking" (not "Health Inc's system").

SECTION 5: DATASET DESIGN (NOVA CONSULTING)

5.1 Scale

Entity	Count
People	~15
Teams	3
Clients	5–6
Projects	8–10
Skills	~15
Documents	60–80

5.2 Document breakdown

Folder	Type	Count	Format
`proposals/`	Client proposals	8–10	Markdown + 2 PDFs
`delivery/`	Delivery notes	8–10	Markdown
`post_mortems/`	What went wrong	5	Markdown
`support/`	Tickets and incidents	12–15	JSON
`slack/`	Simulated channel messages	20–30 messages	JSON (Slack export f
`hr/`	Onboarding docs, policies	3–5	Markdown

Master lists: `people.json` , `clients.json` , `projects.json` , `skills.json` .

5.3 Mandatory test scenarios (non-negotiable)

These are what your evaluation and demo use. Write them deliberately.

Scenario type	Count	Example
Repeated mistake	3	"MongoDB was rejected in 2023 for slow reports" — a new person proposes it again
Who-knows-what	5	"Who knows Kubernetes for banking?"
Cross-client reuse	3	Similar ERP migration done for two clients; reusable approach
Conflicting / superseded decisions	2	Old decision replaced by new one
Multi-hop questions	10+	"Which clients had data quality issues and who led those projects?"
Unanswerable questions	10	Bot should say "I don't know" and route to expert

5.4 How to build the dataset

- **Do not use Lorem Ipsum or generic filler.** Professors and recruiters spot it instantly.
- **Write 10 documents by hand** to set the style.
- **Use Gemini free tier** to draft the rest in the same style.
- **Hand-edit** for consistency: names, dates, prices, technologies must be consistent across documents.
- **Time budget:** 2–3 weeks. This is the foundation. Do not skip it.

5.5 No-Rewrite insurance

Every document's metadata includes from Day 1:

- `tenant_id: "nova"` (for future multi-tenant)
- `access_tags: ["team:delivery"]` (for future permissions)

Unused in Semester 1 but present. Prevents the biggest future rewrite.

SECTION 6: KNOWLEDGE GRAPH SCHEMA (NEO4J)

6.1 Node labels

Label	Key properties
`Person`	id, name, team, role, email, joined_date
`Team`	id, name
`Client`	id, name, industry, size
`Project`	id, name, start_date, end_date, status, type
`Document`	id, title, type, path, date, author_id, team, access_tags, tenant_id
`Decision`	id, text, date, status, decided_by
`Problem`	id, text, severity, first_seen
`Incident`	id, text, date, severity
`Solution`	id, text, applied_date
`Skill`	id, name
`Runbook`	id, title, steps, generated_from

Status values for Decision: proposed / accepted / rejected / superseded.

6.2 Edge types

From	Relation	To	Purpose
Person	`MEMBER_OF`	Team	Team membership
Person	`WORKED_ON`	Project	Assignment
Person	`HAS_SKILL`	Skill	Expertise map
Person	`DECIDED`	Decision	Decision ownership
Person	`SOLVED`	Problem	Who fixed it
Project	`FOR_CLIENT`	Client	Client link
Project	`HAD_PROBLEM`	Problem	What went wrong
Project	`MADE_DECISION`	Decision	Decisions per project
Problem	`SOLVED_BY`	Solution	Fix link
Incident	`OCCURRED_IN`	Project	Where it happened
Incident	`FIXED_BY`	Solution	Fix link
Decision	`SUPERSEDES`	Decision	Old vs new decisions

From	Relation	To	Purpose
Decision	`ABOUT`	Skill	Topic of decision
Document	`MENTIONS`	any node	Provenance
Decision	`EVIDENCED_IN`	Document	Where recorded

6.3 Properties every node must have from Day 1

Property	Used in Sem 1?	Why include now
`tenant_id`	No	Prevents multi-tenant rewrite in business
`access_tags`	Basic only	Prevents permission rewrite in Sem 2
`source_doc_id`	Yes	Provenance for citations
`extracted_at`	No	Audit trail for business
`confidence`	No	Human-in-loop approval later

6.4 Example Cypher queries

Find who knows a skill:

```cypher
MATCH (p:Person)-[:HAS_SKILL]->(s:Skill {name: "Kubernetes"})
RETURN p.name, p.team
```

Find expert for a problem:

```cypher
MATCH (prob:Problem {id: $problem_id})<-[:HAD_PROBLEM]-(proj:Project)<-[:WORKED_ON]-(p:Person)
RETURN p.name, p.team, count(proj) AS experience
ORDER BY experience DESC LIMIT 3
```

Find superseded decisions:

```cypher
MATCH (new:Decision)-[:SUPERSEDES]->(old:Decision)
RETURN old.text AS old_decision, new.text AS new_decision, new.date
```

Multi-hop: clients with a problem and who led them:

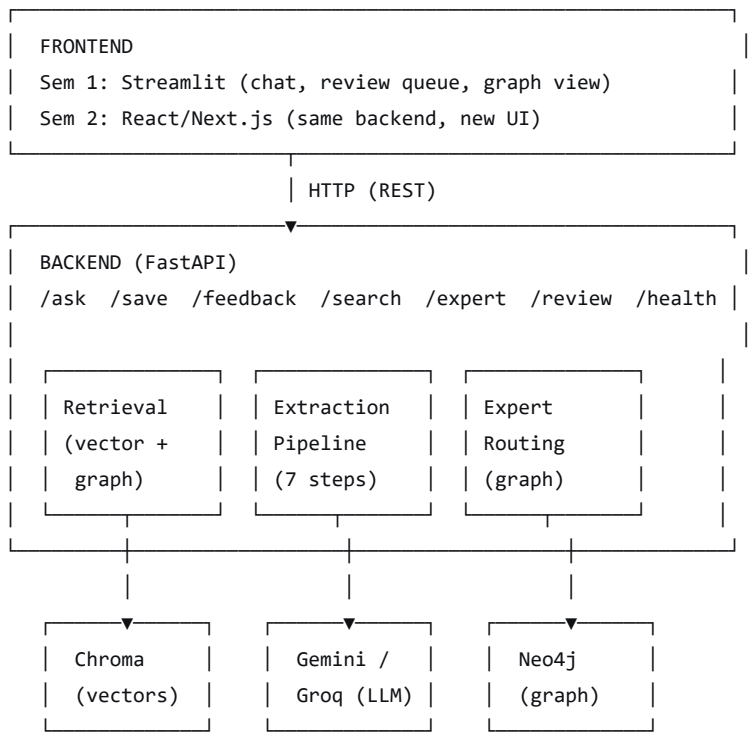
```cypher
```

```
MATCH (c:Client)<-[:FOR_CLIENT]-(proj:Project)-[:HAD_PROBLEM]->(prob:Problem {severity:
"high"})
MATCH (proj)<-[:WORKED_ON]-(p:Person {role: "Tech Lead"})
RETURN c.name, prob.text, p.name
```

SECTION 7: SYSTEM ARCHITECTURE

7.1 High-level diagram (text form)

text



7.2 Free stack (settled)

Need	Choice
LLM	Gemini free tier (primary), Groq free tier (fallback)
Embeddings	sentence-transformers (`all-MiniLM-L6-v2`), local
Vector DB	Chroma (local)
Graph DB	Neo4j Aura Free tier (from Day 1)
Reranker	cross-encoder from sentence-transformers (Sem 2)
Backend	FastAPI
Frontend Sem 1	Streamlit + Streamlit Community Cloud

Need	Choice
Frontend Sem 2	React/Next.js + Vercel free tier
Backend hosting	Render free tier
Connectors	Folder ingestion (Sem 1), Slack + Notion (Sem 2)
Container	Docker (Sem 2 only)
Total cost	**₹0**

7.3 FastAPI routes (the full API surface)

Route	Method	Purpose	Sem
`/health`	GET	Service status	1
`/version`	GET	Version info	1
`/ingest/folder`	POST	Ingest a folder of documents	1
`/search/vector`	POST	Vector search (returns chunks)	1
`/search/graph`	POST	Graph query (returns entities)	2
`/ask`	POST	Main question-answering endpoint	1
`/save`	POST	Save a human answer as memory	1
`/feedback`	POST	Good/bad rating on an answer	1
`/expert`	GET	Find expert for a topic	1
`/review/pending`	GET	List low-confidence extractions	1
`/review/approve`	POST	Approve a pending extraction	1
`/graph/visualize`	GET	Return graph data for visualization	1
`/runbook/generate`	POST	Generate runbook from incident	2
`/onboarding/start`	POST	Onboarding assistant entry	2

7.4 The No-Rewrite Rule in practice

Sem 1 choice	Sem 2 change	Business change
FastAPI routes	Same routes, add auth middleware	Same routes, add tenant isolation

Sem 1 choice	Sem 2 change	Business change
Neo4j schema (with `tenant_id`)	Same schema	Same schema, filter by `tenant_id`
Streamlit calls FastAPI	React calls same FastAPI	React or mobile calls same FastAPI
Folder ingestion	Slack/Notion call same pipeline	More connectors call same pipeline
Chroma (local)	Chroma or swap to pgvector	Swap to managed vector DB
Gemini API	Same or add model router	Same or enterprise LLM

Nothing is thrown away. Everything is upgraded.

SECTION 8: EXTRACTION PIPELINE (DETAILED)

8.1 The 7 steps

```
text

Raw document (PDF / MD / JSON)
    ↓
[1] Loader — read file into text + metadata
    ↓
[2] Chunker — split into meaningful sections
    ↓
[3] Extractor — LLM reads each chunk, outputs structured facts (JSON)
    ↓
[4] Validator — check JSON against Pydantic schema, reject bad output
    ↓
[5] Resolver — match entities to existing nodes (same person? same client?)
    ↓
[6] Writer — create/update nodes and edges in Neo4j (MERGE, not CREATE)
    ↓
[7] Linker — connect facts to source Document node (provenance)
```

8.2 Decisions per step

Step	Decision	Why
Loader	`pymupdf` for PDF, `open()` for Markdown, `json` for Slack exports	Free, reliable, no external service
Chunker	Heading-aware split, then ~500 tokens with 50-token overlap	Preserves structure; overlap prevents cut-off facts

Step	Decision	Why
Extractor	Gemini structured JSON output (with Groq fallback)	Handles messy text; rules cannot parse natural language
Validator	Pydantic models per document type	Prevents garbage entering the graph
Resolver	`rapidfuzz` fuzzy match + LLM only when ambiguous	Cheap, fast, good enough
Writer	Neo4j Python driver, `MERGE` not `CREATE`	Prevents duplicates
Linker	Every fact stores `source_doc_id` and `source_chunk_text`	Required for citations
Retry	3 retries with backoff; fall back to Groq	Free-tier rate limits are real
Human-in-loop	`confidence < 0.7` → review queue	Prevents memory poisoning

8.3 Extraction schema (per document type)

Proposal:

```json
{
  "doc_type": "proposal",
  "client": {"name": "Health Inc", "industry": "healthcare"},
  "project": {"name": "ERP Migration", "type": "data_migration", "start_date": "2024-03-0
1", "duration_weeks": 14},
  "pricing": {"amount": 180000, "currency": "USD"},
  "decisions": [
    {"text": "Chose AWS over Azure", "reason": "existing AWS credits", "decided_by": "Pri
ya", "date": "2024-02-15", "status": "accepted"}
  ],
  "risks_mentioned": [
    {"text": "data quality issues", "severity": "high"}
  ],
  "skills_required": ["data migration", "AWS", "ERP"],
  "confidence": 0.85
}
```

Slack message:

```json
{
  "doc_type": "slack_message",
  "author": "Rahul",
  "channel": "delivery-healthinc",
```

```
  "date": "2024-04-10",
  "decisions": [
    {"text": "Switched to batch processing for consent tracking", "reason": "real-time fa
iled on legacy system", "status": "accepted"}
  ],
  "problems": [
    {"text": "legacy system cannot handle real-time consent tracking", "severity": "hig
h"}
  ],
  "solutions": [
    {"text": "batch processing with 100-record chunks", "applied_to": "consent tracking p
roblem"}
  ],
  "confidence": 0.9
}
```


Support ticket:


```
json

{
  "doc_type": "support_ticket",
  "client": "Retail Corp",
  "reported_by": "client_admin",
  "date": "2025-01-12",
  "incident": {"text": "server restarts every night at 2am", "severity": "high"},
  "solution": {"text": "clear Redis session queue before nightly backup", "applied_by":
"Sanjay"},
  "runbook_steps": ["check Redis memory", "clear session queue", "restart backup job"],
  "confidence": 0.88
}
```


8.4 What the pipeline does NOT do

- Does not extract every sentence — only decisions, problems, solutions, incidents, skills, entities
- Does not trust the LLM blindly — Pydantic validation + confidence threshold + review queue
- Does not build in one pass — batch ingestion, review, re-run

8.5 Time budget

Step	Time
Loader + chunker	3 days
Extractor + schema + validator	1 week
Resolver + writer + linker	1 week
Review queue (Streamlit page)	3 days

Step	Time
Testing on full dataset	1 week
Total	**~4 weeks**

SECTION 9: RETRIEVAL AND ANSWERING

9.1 Semester 1: Standard vector RAG (Baseline B)

Pipeline:

1. User question → embed with sentence-transformers
2. Chroma query → top-5 chunks by cosine similarity
3. Build prompt: question + top-5 chunks + citation instructions
4. Gemini generates answer with `[doc_id]` citations
5. Confidence check: if top similarity score < threshold OR LLM says "not enough evidence," return `escalate: true`

This is the baseline. It works for single-hop questions.

9.2 Semester 2: Hybrid graph + vector retrieval with reranking (System D)

Pipeline:

1. User question → analyze: is it single-hop or multi-hop?
2. **Query planning** (scoped to 2-hop): decompose into sub-questions if needed
3. **Vector retrieval:** top-10 chunks from Chroma
4. **Graph retrieval:** relevant nodes and edges from Neo4j (entity extraction → Cypher)
5. **Merge candidates:** combine vector chunks and graph facts
6. **Rerank:** cross-encoder reranks top-20 → keep top-5
7. **LLM generation** with citations from both sources
8. **Confidence check + expert routing** if low

9.3 Confidence and "I don't know"

Three signals:

Signal	Rule
Retrieval score	If top vector similarity < 0.35, low confidence

Signal	Rule
LLM self-assessment	Prompt asks: "Do you have enough evidence? Answer yes/no and explain"
Graph coverage	If graph has no relevant nodes for the query entities, low confidence

If 2 of 3 say "low," return `escalate: true` with an expert suggestion.

9.4 Expert routing (Sem 1)

When `/ask` escalates:

1. Extract entities from the question (skills, problems, topics)
2. Cypher query: find people connected to those entities
3. Rank by: number of matching connections + recency + team match
4. Return top-3 with reason: "Rahul solved 2 similar problems in Delivery team"

9.5 Save loop (Sem 1)

When a human answers an escalated question:

1. User clicks "Save" or reacts ✅
2. `POST /save` receives: question, answer, expert_id, source_doc_id (optional)
3. Extractor runs on the human answer → produces `Solution` node
4. `Solution` linked to the original `Problem` and to the `Person` who answered
5. Next time a similar question is asked, vector search + graph both find the saved answer

This is the self-growing memory. It is the core differentiator.

SECTION 10: EVALUATION METHODOLOGY

10.1 Test set design

Item	Decision
Total questions	120
Source	60 written manually; 60 LLM-generated + human-verified
Question types	Single-hop factual (40), who-knows-what (20), multi-hop (30), decision/why (20), unanswerable (10)

Item	Decision
Ground truth	Correct answer text, required citation doc IDs, expected expert (if routing), expected behavior (answer vs escalate)
Storage	`eval/questions.json` — versioned, committed to git
Split	60 dev (tune on these), 60 test (only run at end)

10.2 Metrics (exact formulas)

Metric	Formula	Sem 1 Target	Sem 2 Ta
Answer Accuracy	correct / total	≥ 70%	≥ 85%
Citation Precision	answers with correct cited doc / answers given	≥ 90%	≥ 95%
Citation Recall	ground-truth docs cited / total ground-truth docs	≥ 70%	≥ 85%
Hallucination Rate	answers with no supporting citation / answers given	≤ 5%	≤ 3%
Escalation Correctness	(correct escalations + correct non-escalations) / total	≥ 80%	≥ 90%
Expert Routing Top-1	correct expert ranked first / routing events	≥ 60%	≥ 80%
Multi-hop Accuracy	correct multi-hop answers / multi-hop questions	baseline only	≥ 65%
Decision Recall	known decisions retrieved / total tested	baseline only	≥ 75%
Latency p50 / p95	seconds	p50 ≤ 5s, p95 ≤ 12s	p50 ≤ 4s
Cost per query	tokens used × rate	record it	record it

10.3 Four systems compared

System	Description
A. LLM only	Gemini answers without retrieval (baseline)
B. Standard RAG	Vector retrieval + generation
C. RAG + reranking	Vector retrieval + cross-encoder rerank + generation
D. Full system	Vector + graph hybrid + reranking + expert routing + multi-hop

The comparison table (rows = metrics, columns = A/B/C/D) is the **heart of the research**.

10.4 Grading method

- **Automated:** LLM-as-judge (different model from the answering one — e.g., Groq judges Gemini's answers)
- **Human:** 20% random sample manually graded to verify judge
- **Agreement rate:** report it (e.g., "judge agreed with human on 87% of sampled questions")
- **Partial credit:** allowed — 1.0 correct, 0.5 partial, 0 wrong

10.5 What goes in the report

- Full comparison table (Section 9)
- Per-question-type breakdown
- Error analysis: where does the system fail and why
- Latency and cost
- Judge-human agreement rate

SECTION 11: SEMESTER 1 — 16-WEEK PLAN

11.1 Ground rules

- 12–15 hours/week (if more, improve quality, don't add features)
- Git from Week 1, commit daily
- Backend-first always — Streamlit never contains logic
- Neo4j from Day 1
- Free stack only
- No connectors in Sem 1
- No hybrid retrieval in Sem 1

11.2 Week-by-week

Weeks 1–3: Python + skeleton

Week	Python topics	Build	Deliverable
1	Functions, modules, venv, pip, files, JSON, try/except, git	Repo skeleton, README, .env.example, venv	Repo pushe
2	Classes, dataclasses, type hints, Pydantic, .env, requests	FastAPI app with `/health`, `/version`, Pydantic models	`/docs` visit browser

Week	Python topics	Build	Deliverable
3	FastAPI routes, request/response models, uvicorn, async basics	Folder ingestion loader for Markdown + JSON, `POST /ingest/folder`	5 test files i

Checkpoint C1 (end Week 3): FastAPI runs, folder ingestion works.

Weeks 4–6: Chunking, embeddings, vector search

Week	Python topics	Build	Deliverable
4	Text processing, regex, chunking, pymupdf	Chunker module + PDF loader	Chunks with `sourc`
5	sentence-transformers, numpy, cosine similarity	Embedding module + Chroma store + `POST /search/vector`	Top-5 chunks retu
6	Prompt engineering, structured output	RAG answer generation + `POST /ask`	Answer with citati **Baseline B**

Checkpoint C2 (end Week 6): Plain RAG works. If not, stop and fix.

Weeks 7–8: Confidence, "I don't know," graph starts

Week	Python topics	Build	Deliverable
7	JSON mode, confidence scoring	Confidence threshold in `/ask`, `POST /feedback`	"I don't know" + es works
8	Neo4j driver, Cypher (MATCH, MERGE, WHERE)	Neo4j schema setup, populate Person/Team/Project/Client/Document	Graph with 15 peop 8–10 projects

Checkpoint C3 (end Week 8): escalation + graph entities.

Weeks 9–10: Extraction pipeline

Week	Python topics	Build	Deliverable
9	Pydantic nested schemas, retries, backoff	Extractor + validator	10 documents → valid
10	rapidfuzz, entity resolution, MERGE	Resolver + writer + linker	60+ docs extracted; gr populated with decisio problems, solutions, in skills

Checkpoint C4 (end Week 10): populated graph with full provenance.

Weeks 11–12: Expert routing, save loop, review queue, Streamlit v1

Week	Python topics	Build	Deliverable
11	Cypher multi-hop, ranking	Expert routing + `POST /save` + review queue endpoints	Expert suggestion + sa review queue work
12	Streamlit basics	Streamlit v1: chat, citations, escalate card, save button, review page	**MVP loop works end t**

Checkpoint C5 (end Week 12): MVP loop works. If not, cut Weeks 13–14.

Weeks 13–14: Graph visualization + extras

Week	Python topics	Build	Deliverable
13	streamlit-agraph / pyvis	Graph visualization page	Interactive graph c
14	Prompt engineering for runbook + onboarding	Runbook generator + onboarding assistant	Two extra use cas behind)

Weeks 15–16: Evaluation + deployment

Week	Python topics	Build	Deliverable
15	Eval scripts, LLM-as-judge, pandas	60 dev questions + run Baselines A/B + partial D + judge script	`eval/results_sem1.` metrics table
16	Docker basics, Streamlit Cloud, Render	Dockerfile, deploy backend, deploy Streamlit, demo video, README	Live demo + video draft

Checkpoint C6 (end Week 16): deployed, evaluated, demoable.

11.3 Checkpoints

ID	Week	Pass condition	If fail
C1	3	FastAPI runs, ingestion works	Spend Week 4 fixing
C2	6	Ask → answer + citations	Stop, fix, do not procee
C3	8	Escalation + graph entities	Simplify graph
C4	10	60+ docs extracted	Reduce fact types
C5	12	MVP loop works	Cut Weeks 13–14

ID	Week	Pass condition	If fail
C6	16	Deployed + evaluated	Local demo + smaller ev

SECTION 12: SEMESTER 2 — FULL PLAN

12.1 Goal

Turn the MVP into a research-grade major project with a proper comparison, real connectors, a polished web app, and a user study.

12.2 Time allocation

- **70% research** (graph, hybrid retrieval, multi-hop, evaluation, user study)
- **30% product** (web app, connectors, polish)

Do not invert this. A pretty app with weak research fails. A strong research system with a decent UI passes.

12.3 Features to build

Feature	Why	Time
Knowledge graph (full)	Research core	3 weeks
Graph + vector hybrid retrieval + reranking	Fair comparison architecture	2 weeks
Multi-hop reasoning (scoped 2-hop)	The measurable win	4–6 weeks
Full 4-system evaluation	The paper's heart	2 weeks
User study (5–10 people, 2 weeks)	Real validation	3 weeks
Slack connector	Real "present" data	1–2 weeks
Notion connector	Real "past" data	1 week
Runbook generator (polished)	Support value	1 week
Onboarding assistant (polished)	New-hire value	1 week
React/Next.js web app	Production UI	3 weeks
Docker deployment	Resume-worthy	3 days
Role-based access (5 levels, 3 teams)	Business realism	1 week

Feature	Why	Time
Proactive warning ("this was tried before")	Original idea as a feature	2 weeks
Full report / paper	Deliverable	ongoing

12.4 Semester 2 milestones

Milestone	When	Deliverable
M1: Graph complete + hybrid retrieval	Week 4	All nodes/edges populated; hybrid retrieval working
M2: Multi-hop (scoped) working	Week 8	2-hop questions answered via graph
M3: Full evaluation run	Week 10	4-system comparison on 120 questions
M4: Web app deployed	Week 12	React app on Vercel, backend on Render
M5: Connectors working	Week 12	Slack + Notion ingesting
M6: User study complete	Week 13	5–10 users, feedback collected
M7: Final report	Week 16	40–60 page report + demo video

12.5 User study design

- **Participants:** 5–10 people (classmates, friends, a professor)
- **Duration:** 2 weeks
- **Tasks:** ask 10 questions each, rate answers, note failures
- **Data collected:** accuracy perception, trust, time saved, would-use-again
- **Analysis:** qualitative + simple quantitative (average rating, top complaints)
- **Report section:** "User Study" in Section 10 of the report

12.6 MCP placement

Skip MCP for college. One paragraph in Future Work.

Reason	Detail
Not a research contribution	Distribution protocol, not capability
Costs 1–2 weeks	No evaluation value

Reason	Detail
Confuses demo audience	Nobody in college knows MCP
Sem 2 time is precious	Goes to hybrid retrieval and multi-hop

Report paragraph (Section 11, Future Work):

> "The system exposes a REST API following the OpenAPI standard. A natural next step is to wrap these endpoints as a Model Context Protocol (MCP) server, allowing AI tools such as Claude Desktop or Cursor to query organizational memory directly. This would extend the system from a human-facing tool to an AI-facing knowledge service."

SECTION 13: BUSINESS ROADMAP (FUTURE — DO NOT BUILD NOW)

13.1 Five depth areas

1. Extraction quality

- Human-in-loop approval at scale
- Duplicate merging
- Conflict detection (new decision contradicts old)
- Entity resolution at scale
- Confidence scoring

2. Graph depth

- Temporal graph (decisions with validity periods)
- Multi-hop query planning (beyond 2-hop)
- Provenance tracking (every fact → source)
- Graph-based recommendation ("projects similar to this")

3. Connectors

- Google Drive, Notion, Confluence
- Slack, Microsoft Teams
- Jira, Asana, Monday
- GitHub, GitLab
- Email (Gmail, Outlook)
- CRM (Salesforce, HubSpot)
- Meeting transcripts (Zoom, Gong)

- Each with incremental sync + permission mapping

4. Security and compliance

- SSO / SAML / OAuth
- Fine-grained RBAC (per team, per client, per document)
- Permission-aware retrieval
- Full audit log
- Data residency options
- PII detection and redaction
- Encryption at rest and in transit
- Retention policies
- GDPR / SOC 2 readiness

5. Operations and scale

- Multi-tenant isolation (already prepared via `tenant_id`)
- Background job queue (Celery / Redis)
- Caching layer
- Rate limiting
- Cost tracking per query
- Model routing (cheap model for simple, strong for hard)
- Monitoring, alerting, error tracking
- Admin console with usage analytics

13.2 Packaging

Tier	Price	Features
Free	₹0	1 workspace, 100 queries/month
Pro	~$8/user/month	Slack/Teams, connectors, analytics
Business	~$15–20/user/month	SSO, RBAC, audit, API, MCP
Enterprise	Custom	Self-hosted, VPC, SLA, support

13.3 Distribution

- SaaS app first (fastest to sell)
- Then self-hosted (for security-conscious companies)
- Then API + MCP (for AI-native teams)
- Open-core is a good growth strategy

13.4 Honest note

The business version is 10x the work. Sell the idea on the strength of the research and demo, not by building all features. Most commercial features are connectors and compliance — unglamorous but that is what companies pay for.

SECTION 14: DEMO SCRIPT (5 MINUTES)

14.1 The structure

0:00–0:30 — Problem in one sentence

Say: "In consulting companies, knowledge is lost. A proposal that took 3 days should take 3 hours. A mistake made in 2024 is repeated in 2026. This system fixes that."

Show: slide with "3 days → 3 hours" and "repeated mistake → prevented."

0:30–1:30 — Wow #1: Answer with citations

Ask: "Have we done a legacy ERP migration before?"

Show: three past projects with client, duration, price; every claim cited; common risk highlighted.

Say: "Not a chatbot guessing. Every fact traces to a real document."

1:30–2:30 — Wow #2: "I don't know" + expert routing

Ask a question the system cannot fully answer.

Show: system says "I don't have enough information"; it does not guess; it says "Sunita led two similar projects. Ask her?" Click → shows her work.

Say: "Most AI hallucinates here. Ours admits ignorance and finds the right person."

2:30–3:30 — Wow #3: The graph

Open graph page. Click "Data Quality." Show it expanded: 3 projects, 2 clients, 4 people, solution, expert.

Say: "This is why we use a knowledge graph. Normal search finds one document. The graph finds connections."

3:30–4:15 — The quiet wow: learning loop

Ask an unknown question → escalate → simulate human answer → save → ask again → answered instantly.

Say: "The memory grows by itself. Nobody maintains a knowledge base by hand."

4:15–5:00 — The numbers, then stop

Show evaluation table:

System	Answer Accuracy	Multi-hop Accuracy	Hallucination Rat
LLM only	42%	22%	31%
Standard RAG	61%	34%	9%
Our system	**78%**	**66%**	**4%**

Say: "Tested on 120 real questions. Every number is reproducible; the test set is in the repo."

Then stop. Let the numbers land.

14.2 Demo rules

- Never debug live
- Wake free-tier backend 5 minutes before
- Have a recorded backup video
- Pre-load the graph page
- End on numbers, not on a feature

SECTION 15: REPORT STRUCTURE

Target: 40–60 pages. Use this exact structure.

Section	Pages	Content
1. Abstract	1	Problem, system, method, key result (write last)
2. Introduction	3–4	Knowledge-loss problem, gap in existing tools, 3-bullet contribution
3. Literature Review	5–6	RAG, GraphRAG, organizational memory, expert routing, agentic systems, 15–25 citations
4. Problem Statement & RQs	1–2	RQ1–RQ4 from Section 3
5. System Design	8–10	Architecture diagram, 7-step pipeline, Neo4j schema, FastAPI routes, free stack
6. Dataset	3–4	Nova Consulting dataset, document breakdown, test scenarios, build method (this is a contribution)

Section	Pages	Content
7. Implementation	6–8	Chunking, embeddings, retrieval, extraction, entity resolution, expert routing, save loop, graph visualization (key snippets only)
8. Evaluation Methodology	3–4	120-question test set, 5 types, 10 metrics with formulas, LLM-as-judge + human verification, 4 systems
9. Results	4–5	Comparison table, per-type breakdown, error analysis, latency, cost
10. Discussion	2–3	Why GraphRAG wins, latency cost, limitations, threats to validity
11. Conclusion & Future Work	1–2	Summary, Sem 2 additions, business path, MCP paragraph
12. References	2	25–40 references
Appendix	3–5	Full graph schema, sample extractions, sample questions, setup

15.1 Report rules

- Write Section 5 and 8 before coding finishes
- Write Section 9 immediately after evaluation
- Every claim needs a number or a citation
- Limitations section is maturity, not weakness
- The evaluation table in Section 9 is the single most important page

SECTION 16: RISK AND FALLBACK PLAN

Risk	When	Fallback
Python harder than expected	Weeks 1–3	Spend Week 4 on Python only; cut Week 14 extras
Plain RAG not working	Week 6	Stop. Debug chunker, embeddings, prompt. Do not proceed.

Risk	When	Fallback
LLM extraction poor	Weeks 9–10	Extract fewer fact types (Decision + Problem + Solution); improve prompt with 3 examples; review queue catches rest
Neo4j confusing	Week 8	Use Aura free tier browser UI; test queries there first; Neo4j Community in Docker as backup
Gemini rate limits	Any week	Switch to Groq; keep both keys in `.env` ; wrap LLM in `llm.py` module
Tempted by connectors	Weeks 11–16	Say no. Folder ingestion is enough for Sem 1.
Behind schedule at Week 12	Week 12	Cut Week 14 (runbook + onboarding). Keep Week 13 (graph viz). Keep Week 15 (eval).
Evaluation takes too long	Week 15	Reduce test set from 120 to 60 (40 dev, 20 test)
Deployment fails	Week 16	Demo locally on video. Acceptable.
Multi-hop too hard	Sem 2	Scope to fixed 2-hop templates. Do not attempt in Sem 1.
Loss of motivation	Any week	Week 12 MVP is the anchor. Commit daily even if small.
Scope creep (new idea)	Any week	Write it in `docs/ideas.md` . Do not build it.

Golden rule: if behind at Week 12, cut everything except the MVP loop. A working MVP with honest evaluation beats a half-built ambitious system.

SECTION 17: THE SEVEN RULES

Tape these to your wall.

1. **Backend-first, always.** Streamlit is disposable. FastAPI is forever.
2. **Neo4j from Day 1.** No schema swaps.
3. `tenant_id` **and** `access_tags` **on every node and document, even when unused.** This is No-Rewrite insurance.
4. **Every fact traces to a source document.** No provenance, no fact.
5. **Evaluation is designed before it is run.** Test set written before system is finished.

6. **Working MVP beats ambitious half-system.** At Week 12, the loop must work.
7. **Scope creep is the enemy.** New ideas go in `docs/ideas.md`, not into the build.

SECTION 18: WHAT YOU DO ON MONDAY

1. Create GitHub repo with folder structure:

```
text

nova-memory-agent/
├── backend/
├── frontend/
├── data/
├── eval/
├── docs/
├── README.md
└── .env.example
```

2. Write `README.md` with the one-line pitch and free stack.
3. Write `docs/plan.md` — paste this entire document into it.
4. Start Week 1: Python fundamentals, virtual environment, first FastAPI `/health` route.
5. Commit before you sleep.

SECTION 19: FINAL HONEST SUMMARY

What you have now:

- A clear problem (knowledge loss in consulting)
- A novel combination (self-growing memory + honest ignorance + expert routing + GraphRAG)
- A decided vertical (service-based consulting, Nova)
- A decided schema (Neo4j, 11 node types, 15 edge types)
- A decided pipeline (7 steps, LLM extraction, confidence + review queue)
- A decided evaluation (120 questions, 10 metrics, 4 systems)
- A decided stack (₹0, Gemini + sentence-transformers + Chroma + Neo4j Aura + FastAPI + Streamlit + Render)
- A decided 16-week plan (with 6 checkpoints)
- A decided Sem 2 plan (70% research, 30% product)
- A decided demo (5 minutes, 3 wow moments)
- A decided report structure (12 sections, 40–60 pages)
- A decided risk plan (12 risks, pre-decided fallbacks)
- A business roadmap (5 depth areas, 4 tiers)

What you need to do:

Build it. Week by week. Commit daily. Hit checkpoint C2 at Week 6 (plain RAG works) and C5 at Week 12 (MVP loop works). Everything else follows.

The project is real. The plan is complete. The stack is free. The research is defensible.

Start Week 1 on Monday. Do not redesign again.

> i want this response save in document of word or docs how can i by directly pasting all tabel and all not pasting correctly . give ans in short

Thought for 2 seconds

> The user wants to save my full plan response into a Word or Google Docs document, but pasting directly messes up tables. They want a short answer on how to do it.
>
> Let me give practical solutions.
>
> Options:
>
> 1. Copy from the chat and paste into Google Docs/Word - tables sometimes break
> 2. Use a markdown-to-docx converter
> 3. Ask me to reformat as HTML or something pasteable
> 4. Use print-to-PDF from browser
> 5. Use a tool like "Markdown to Word" converter
>
> Best approaches:
>
> - Select all → copy → paste into Google Docs (Docs usually handles tables from markdown better than Word if you use "paste from markdown")
> - Better: save as .md file, then use pandoc or an online converter
> - Or: I can give them the content as HTML that they can save as .html and open in Word
> - Or: Use browser print (Ctrl+P) → Save as PDF, then convert to Word
>
> Actually the cleanest for them:
>
> 1. Copy text from chat
> 2. Paste into a plain text editor (.md file)
> 3. Use an online markdown-to-docx converter (like cloudconvert, or dillinger.io)
>   OR
> 4. Use browser print to PDF
> 5. Open PDF in Word (Word converts PDF to editable doc with tables)
>
> Let me give 3 short options ranked by ease.
>
> Keep it short as they asked.

Short answer — 3 ways, easiest first:

Option 1 (best): Markdown → Word converter

1. Copy my full response from chat.
2. Paste into a plain `.md` file (Notepad / VS Code).
3. Go to **dillinger.io** or **cloudconvert.com/md-to-docx** → upload → download `.docx`.
4. Tables, headings, code blocks all come through clean.

Option 2 (no tools): Browser print → PDF → Word

1. In the chat, press `Ctrl+P` (print).
2. Save as **PDF**.
3. Open the PDF in **Microsoft Word** → it auto-converts tables into editable Word tables.

Option 3 (Google Docs specifically):

1. Copy my response.
2. In Google Docs: **Paste special** → **Paste from Markdown** (or just paste, then `Format → Clear formatting`).
3. If tables break: use Option 1, then upload the `.docx` to Google Drive and open with Google Docs.

Fastest reliable combo: Option 1 (dillinger.io) → download `.docx` → open in Word or upload to Google Docs. Tables survive.